Excel for Finance

The Complete Professional Handbook

Module 2

Making Excel Professional

Optimisation, Macros & VBA, Absolute/Relative References,
Named Ranges, Data Validation — with Python and SQL Equivalents

Companion modules in this handbook:

- **Module 1** — Excel Foundations: Interface, Data & Number Formatting, Formula Fundamentals, Shortcuts
- **Module 2** — Making Excel Professional: Optimisation, Macros, Absolute/Relative References, Named Ranges, Data Validation & Custom Formatting *(this document)*
- **Module 3** — Excel Like a Pro: Conditional Formatting, Logical Functions, Lookup Functions (VLOOKUP/HLOOKUP/XLOOKUP/INDEX-MATCH), Conditional Aggregation, Error Handling, String Functions

Compiled study notes covering the Excel curriculum end-to-end, extended with equivalent Python (`pandas`, `openpyxl`) and SQL implementations of every concept, for a tech-background learner building cross-tool fluency in financial analytics.

August 15, 2026

Contents

1 How to Use This Module **1**

2 From "Works" to "Professional": What This Module Adds **1**

3 Optimising and Professionalising a Workbook **2**
3.1 Two Different Meanings of "Optimisation" . . . 2
3.2 Wrap Text vs. the Manual Line Break . . . 2
3.3 Text to Columns: Splitting One Cell of Text into Many . . . 3
3.4 The Recurring Weakness of Text to Columns . . . 4
3.5 Removing Visual Clutter: Gridlines and Headings . . . 4
3.6 Row Height and Column Width . . . 5
3.7 Fonts: Consistency Over Decoration . . . 5
3.8 Excel Options: Configuring the Application Itself . . . 5
3.9 Opening a Specific Workbook Automatically at Startup . . . 7
3.10 Calculation Mode and Performance Optimisation . . . 7
3.11 Cross-Tool Translation: Python and SQL . . . 9

4 Macros and VBA: Automating Repetitive Tasks **10**
4.1 What a Macro Actually Is . . . 10
4.2 Enabling the Developer Tab . . . 10
4.3 Recording a Macro . . . 11
4.4 Saving Macros: the .xlsm File Format . . . 11
4.5 Macro Security . . . 12
4.6 Reading and Editing Recorded VBA Code . . . 12
4.7 The "Only Works for 4 Rows" Problem, Solved . . . 13
4.8 Writing a Macro From Scratch: Basic VBA Building Blocks . . . 14
4.9 The Personal Macro Workbook . . . 15
4.10 Running a Macro: Buttons, Shapes, and the Quick Access Toolbar . . . 15
4.11 The Real Limitations of Macros . . . 15
4.12 Cross-Tool Translation: Python and SQL . . . 16

5 Absolute vs. Relative Cell References **18**
5.1 Recap: Why This Matters . . . 18
5.2 The $ Symbol: Locking a Reference . . . 18
5.3 The F4 Cycling Shortcut . . . 19
5.4 Mixed References in Practice: A Two-Way Sensitivity Grid . . . 19
5.5 References Across Sheets and Workbooks . . . 20
5.6 Cross-Tool Translation: Python and SQL . . . 21

6 Named Ranges **22**
6.1 Why Named Ranges Exist . . . 22
6.2 Creating a Named Range . . . 22
6.3 Naming Rules . . . 22
6.4 Using a Named Range in a Formula . . . 23
6.5 Scope: Workbook vs. Worksheet . . . 23
6.6 The Name Manager . . . 23
6.7 Named Ranges as Data Validation List Sources . . . 23
6.8 Dynamic Named Ranges and the Modern Alternative: Excel Tables . . . 24
6.9 Named Constants: a Name With No Cell At All . . . 24
6.10 Common Pitfalls . . . 25
6.11 Cross-Tool Translation: Python and SQL . . . 25

7 Data Validation and Custom Validation Formulas **26**
7.1 Why Data Validation Exists . . . 26
7.2 The Data Validation Dialog: Three Tabs . . . 26
7.3 The Settings Tab: Allow Types . . . 27
7.4 List Validation: Building a Dropdown . . . 27
7.5 Whole Number, Decimal, Date, and Text Length Validation . . . 28
7.6 Custom Validation: Writing Your Own Rule . . . 28
7.7 Input Message and Error Alert . . . 29
7.8 Circling Invalid Data . . . 30
7.9 Copying, Extending, and Removing Validation . . . 30
7.10 Cross-Tool Translation: Python and SQL . . . 30

8 Worked Example: A Professional Assumptions Sheet in Three Tools **32**
8.1 The Brief . . . 32
8.2 Excel Implementation . . . 32
8.3 Python Implementation . . . 33
8.4 SQL Implementation . . . 34

9 Common Pitfalls & Debugging Checklist **35**

10 Chapter Summary & Self-Check **35**

11 What's Next: Module 3 Preview **36**

1 How to Use This Module

This module assumes you have completed Module 1 and can comfortably navigate the Excel interface, format numbers, and write basic formulas with relative references. Everything here builds directly on that foundation. As in Module 1, every topic is explained three times, in three dialects:

- **Excel** — the native implementation: menu paths, dialog boxes, and keyboard shortcuts, exactly as you would click through them.
- **Python** — the equivalent using `openpyxl` (for building/formatting actual `.xlsx` files programmatically) and `pandas` (for the data-manipulation mindset that replaces several Excel features entirely).
- **SQL** — the closest conceptual equivalent, since a surprising number of "make Excel professional" features exist to compensate for the fact that a worksheet has none of a database's built-in structure, constraints, or automation.

The same recurring boxes from Module 1 appear throughout:

Key Concept

A foundational idea you must internalise before moving on — the "why", not just the "how".

Finance Application

How the concept is specifically used in financial modeling, accounting, or investment analysis.

Pro Tip

A workflow habit that separates a fast, professional analyst from a slow, error-prone one.

Common Pitfall

A mistake that looks harmless but silently corrupts a financial model.

Formula Syntax

The precise syntax of a function, code, or dialog setting, written the way Excel itself would present it.

2 From "Works" to "Professional": What This Module Adds

Module 1 got a workbook to *produce correct numbers*. That is necessary but not sufficient. A workbook used in a real finance role also has to survive contact with other humans: a colleague who inherits it six months from now, an auditor who has to trust it, a repetitive monthly task that must not eat an hour every time, and a user who will absolutely, eventually, type the wrong thing into an input cell. This module is about the five specific skills that take a spreadsheet from "technically correct" to "professional":

1. **Optimisation** — a workbook that is fast, clean, and configured sensibly, rather than sluggish and visually chaotic.
2. **Macros** — automating a repetitive manual task instead of repeating it fifty times by hand.
3. **Absolute vs. Relative References** — the full `$` notation, so formulas copy correctly instead of subtly breaking.
4. **Named Ranges** — replacing fragile, meaningless cell addresses (`$B$4`) with self-documenting names (`TaxRate`), directly solving the positional-reference fragility flagged in Module 1.

5. **Data Validation** — constraining what a user is allowed to type into a cell, Excel's closest equivalent to a database `CHECK` constraint.

The Common Thread

Every topic in this module exists to solve the same underlying problem: **a spreadsheet has no enforced structure**. A database table rejects a row that violates a constraint; a Python function signature rejects the wrong type of argument; a compiled program will not even build if a reference is invalid. A blank Excel cell, by contrast, will accept literally anything, and a formula will happily reference the wrong cell forever without complaint. Named Ranges, Data Validation, and disciplined use of absolute references are three different techniques for manually re-imposing the structure that other tools give you for free. Macros and workbook optimisation are about the other half of "professional": performance and repeatability at scale, without the safety net of code being run from source control by a CI pipeline.

3 Optimising and Professionalising a Workbook

3.1 Two Different Meanings of "Optimisation"

The course label "Optimisation" actually bundles two distinct goals that are worth separating in your head, because they use different tools:

Cosmetic Optimisation vs. Performance Optimisation

Cosmetic optimisation makes a workbook *look* clean, readable, and deliberate — consistent fonts, no stray gridlines, sensibly sized columns, no visual clutter. This is about *trust*: a messy-looking model makes a reader doubt the numbers even when they're correct. **Performance optimisation** makes a workbook *run* fast even as it grows to thousands of formulas — calculation mode, avoiding volatile functions, avoiding whole-column references. This is about *usability*: a workbook that takes eleven seconds to recalculate after every keystroke will not get used carefully, however elegant its formulas are. A genuinely professional workbook needs both; this section covers each in turn, plus the Excel Options settings that control both.

3.2 Wrap Text vs. the Manual Line Break

Module 1 introduced **Wrap Text** (Home tab) as a way to keep long text inside one cell by increasing row height instead of overflowing into the next cell. There is a second, related tool worth knowing precisely:

Manual Line Break

`Alt + Enter` while editing a cell inserts a line break *at the cursor position*, forcing a new line exactly where you choose — independent of column width. It also automatically turns on Wrap Text for that cell, since the line break would otherwise be invisible.

Tool	Breaks the line	Use when
Wrap Text (automatic)	Wherever the column width forces it, re-flowing if the column is resized	The text is one continuous sentence/paragraph and you want it to always fit the current column width
`Alt+Enter` (manual)	Exactly where you place the cursor, permanently	You are building a multi-line label with deliberate structure, e.g. `Revenue (USD, 000s)` on two fixed lines regardless of column width

Finance Application

Two-line column headers — a metric name on the first line and its unit on the second (`"EBITDA Margin"` / `"(%)"`) — are a near-universal convention in financial models and are built with `Alt+Enter`, not by hoping Wrap Text breaks the line in the right place.

3.3 Text to Columns: Splitting One Cell of Text into Many

Text to Columns (**Data** tab → **Text to Columns**) takes a column where each cell contains multiple pieces of information glued together as one text string, and splits it into separate columns — the single most common "data came in messy" fix in Excel.

Text to Columns Wizard — Three Steps

1. **Original data type**: **Delimited** (split wherever a character like a comma, tab, space, or semicolon occurs) or **Fixed width** (split at fixed character positions you click to define — for data with no separator character, like a legacy fixed-format bank export where columns 1–10 are always the account number).
2. **Delimiters** (Delimited only): choose which character(s) mark a split — Tab, Semicolon, Comma, Space, or a custom character — and optionally treat consecutive delimiters as one (important for text with irregular spacing).
3. **Column data format**: for each resulting column, choose **General**, **Text**, or **Date** (with the date order, e.g. DMY vs. MDY) — and a **Destination** cell for where the split output should start (default overwrites the original column).

Before (one cell)	Delimiter	After (three cells)
`Sharma, Anjali, Mumbai`	Comma	`Sharma` | `Anjali` | `Mumbai`
`INV-2026-00451`	Custom: `-`	`INV` | `2026` | `00451`
`2026-08-15` (as text)	— (Fixed width or Date column type)	Recognised and re-parsed as an actual date value

The "Silent Fix" for Numbers Stored as Text

Module 1 flagged **numbers stored as text** as a silent data-quality trap: such cells look like numbers but are excluded from `SUM`/`AVERAGE` without any error. Running **Text to Columns** on the affected column with **Delimited** selected but *no* delimiter checked, then clicking **Finish**, forces Excel to re-parse every cell's content and convert genuine numeric text back into real numbers — a one-click fix that requires no formula at all.

3.4 The Recurring Weakness of Text to Columns

Text to Columns is a *one-time, one-range* operation: you select a specific range, run the wizard once, and it acts only on the cells you had selected at that moment. It is not a live, self-updating feature the way a formula is — if five new rows of comma-separated data are pasted in below tomorrow, they will *not* automatically get split; you have to select them and re-run the wizard.

Why a Recorded "Text to Columns" Macro Only Works for a Fixed Number of Rows

This limitation is the exact reason a *macro* that records a Text to Columns action (§4) breaks the moment your dataset grows: the macro recorder captures the literal range you had selected while recording — say `A1:A4` — and writes that fixed address into the generated code. Run the same macro next month on a sheet with 40 rows, and it will still only touch `A1:A4`, silently leaving rows 5–40 unsplit, with no error or warning. Section 4 shows exactly how to fix this by editing the recorded code to use a dynamic range instead of a hard-coded one.

3.5 Removing Visual Clutter: Gridlines and Headings

Module 1's View tab section introduced toggling **Headings** and **Gridlines** on or off. It is worth being precise about *why* a professional deliverable almost always has them off:

- **Gridlines** (**View** tab → uncheck **Gridlines**) are a screen aid for building a sheet, not part of the finished product. A dashboard, summary report, or printed statement with gridlines visible looks like an unfinished worksheet rather than a designed document — professional models instead use deliberate **borders** (Home tab → Borders) only where a visual boundary is actually meaningful (under a subtotal, around an input box).
- **Row/Column Headings** (the grey `A B C...` / `1 2 3...` bars) are similarly a building aid. Turning them off (same **View** tab checkbox) is standard for a polished dashboard tab, though they are normally left *on* for a working calculation tab where you still need to reference cell addresses while auditing.
- These two settings are **per-worksheet**, not per-workbook — a common workbook pattern is gridlines/headings *off* on a front-facing "Dashboard" or "Summary" tab, and *on* for the underlying "Calculations" or "Data" tabs the analyst still works in directly.

Screen Gridlines vs. Printed Gridlines Are Two Separate Settings

Turning off **View** → **Gridlines** only affects what you see on screen. Whether gridlines appear on a *printed* page (or a PDF export) is a separate checkbox: **Page Layout** tab → **Sheet Options** → **Gridlines** → **Print**. It is entirely possible (and a common mistake) to have a clean-looking screen that still prints with gridlines, or vice versa.

3.6 Row Height and Column Width

Method	How	Best for
Auto-fit by double-click	Double-click the border between two column letters (or row numbers)	Quickly sizing a column to exactly fit its widest current entry
Manual exact size	Right-click a column/row header → **Column Width**/**Row Height** → type a number	Making several columns *identically* sized for a clean grid look (auto-fit alone rarely produces matching widths)
Format menu	**Home** → **Format** → **AutoFit Column Width** / **AutoFit Row Height**	Applying auto-fit to a multi-column/row selection at once

Standard Widths for Financial Models

Many banking and equity-research templates set every input/label column to a uniform width (a common convention is roughly 9–11 characters, i.e. just wide enough for a value like `(1,234.5)`) rather than auto-fitting each column independently — the uniformity itself is what reads as "professional", even more than the exact number chosen.

3.7 Fonts: Consistency Over Decoration

Key Concept

Unlike a Word document or a slide deck, a spreadsheet's font choice is not primarily about aesthetics — it is about **numeric legibility and consistency**. A single font, single size, applied uniformly, is what allows a reader's eye to scan a column of numbers instantly for outliers. Calibri and Arial are the two overwhelmingly dominant choices in professional finance because both render numerals with clearly distinct digit shapes and consistent character width at small sizes; a workbook mixing three or four different fonts reads as unplanned, however individually reasonable each font is.

A typical convention: one base font (e.g. Arial 10 or Calibri 11) for the entire workbook, with **bold** reserved for headers/totals and **font colour** reserved for the input-cell convention (blue = hard-coded input, black = formula, green = link to another sheet) rather than for decoration.

3.8 Excel Options: Configuring the Application Itself

Everything so far has configured a *worksheet*. **File** → **Options** configures *Excel itself*, and several settings here directly affect how professional and efficient your day-to-day workbook building is.

Tab	Setting	Why it matters
General	Default font & size for new workbooks	Locks in the font-consistency convention above at the source, instead of reformatting every new workbook by hand.
Formulas	Workbook Calculation: Automatic / Automatic Except for Data Tables / Manual	Covered in depth in §3.10 — the single biggest lever for performance on a large model.
Formulas	R1C1 reference style	Switches cell addressing from `A1`-style to `R1C1` (Row 1, Column 1) notation — rarely used day-to-day, but this *is* the addressing style VBA's recorded code uses internally (§4), so recognising it helps when reading macro code.
Formulas	Enable iterative calculation	Required to allow deliberate **circular references** (Module 1, §"Common Pitfalls") to resolve instead of erroring — used sparingly, e.g. for a debt schedule where interest expense depends on a cash balance that itself depends on interest expense.
Save	AutoRecover interval; default file format	Set the autosave interval low (e.g. 5 minutes) for large models where a crash is costly; default format should be `.xlsx` (or `.xlsm` if the workbook always carries macros, §4).
Advanced	After pressing Enter, move selection	Direction the active cell moves after `Enter` — set to **Down** for entering a column of data quickly, or disabled entirely for formula auditing.
Advanced	Show a zero in cells that have a zero value	Unchecking this displays zero-value cells as blank — common in presentation-ready financial statement tabs where a literal `0` is visually noisier than empty space.
Customize Ribbon	Developer tab checkbox	**Required** to access macro recording, the VBA editor, and Data Validation's more advanced options — covered next in §4.
Trust Center	Macro Settings	Controls whether macros run automatically, prompt, or are blocked outright when a workbook is opened — see the security note in §4.5.

3.9 Opening a Specific Workbook Automatically at Startup

The XLSTART Folder

Any workbook, template, or workspace file placed inside Excel's **XLSTART** folder is opened automatically, every time Excel launches. On Windows, the default per-user location is:

`C:\Users\<username>\AppData\Roaming\Microsoft\Excel\XLSTART`

An **alternate startup folder** can additionally be specified at **File → Options → Advanced → General → "At startup, open all files in:"**, useful when the default AppData path is inconvenient to reach or you want a shared, version-controlled startup folder.

This is also precisely how the **Personal Macro Workbook** (`PERSONAL.XLSB`) makes your macros available in *every* workbook you open, not just the one they were recorded in — it is simply a workbook that lives in `XLSTART` and is therefore always loaded (see §4.9).

Auto-Run Macros on Startup

If a workbook placed in `XLSTART` contains a macro named exactly `Auto_Open`, that macro executes automatically every time Excel starts — powerful for setting up a standard environment, but also the reason security software and Excel's own Trust Center are cautious about auto-opened, macro-enabled files from an untrusted source (§4.5).

3.10 Calculation Mode and Performance Optimisation

As a model grows past a few thousand formulas, *when* Excel recalculates starts to matter as much as *what* it calculates.

Calculation Modes — Formulas Tab → Calculation Options

- **Automatic** (default): every formula in every open workbook recalculates after *any* change, anywhere.
- **Automatic Except for Data Tables**: everything recalculates automatically *except* What-If Analysis Data Tables, which are often the single most expensive object in a model (they can force thousands of hidden recalculations) — a good middle ground.
- **Manual**: nothing recalculates until you explicitly trigger it with `F9` (calculate all open workbooks), `Shift+F9` (calculate only the active sheet), or `Ctrl+Alt+F9` (force a full recalculation of everything, ignoring Excel's usual "only recalculate what changed" optimisation — useful when you suspect a stale cached value).

When to Switch to Manual

On a large financial model (multi-tab DCFs, three-statement models, anything with thousands of formulas or several Data Tables), switching to **Manual** calculation while actively editing assumptions — then pressing `F9` deliberately once changes are finished — can turn a multi-second freeze after every keystroke into an instant, responsive editing experience. The discipline this requires (remembering to recalculate before trusting a displayed number) is the trade-off; Excel shows **"Calculate"** in the status bar as a reminder that a recalculation is pending.

Volatile Functions

A **volatile** function forces *itself and everything that depends on it* to recalculate on *every single* recalculation event, even when none of its actual inputs changed. The most common volatile functions are `NOW()`, `TODAY()`, `RAND()`, `RANDBETWEEN()`, `OFFSET()`, `INDIRECT()`, and `CELL()`/`INFO()` in some argument forms. A model with a few hundred `OFFSET`-based dynamic ranges can become dramatically slower than an equivalent model built with `INDEX` (which is *not* volatile and returns a reference just as flexibly) — a swap worth knowing about even before Module 3 formally introduces `INDEX`/`MATCH`.

Other Performance Habits

Beyond calculation mode: avoid referencing *entire columns* (`=SUM(A:A)`) when a bounded range (`=SUM(A2:A10000)`) will do, since a whole-column reference forces Excel to consider all 1,048,576 rows on every recalculation; prefer helper columns with simple formulas over one deeply nested formula repeated across a sheet (simple formulas repeated are cheaper for Excel's calculation engine, and dramatically easier for a human to audit); and keep formatting (especially conditional formatting rules) as narrow as the actual data range rather than applied to whole rows/columns "just in case".

3.11 Cross-Tool Translation: Python and SQL

Excel concept	Python equivalent	Notes
Manual calculation mode	Simply not re-running a script until you choose to	Python has no equivalent of an "always-on reactive engine" to switch off — a `.py` script only computes when you execute it, so this entire lever is unnecessary in Python; it exists in Excel purely because Excel recalculates by default.
Avoiding volatile functions	Avoiding an expensive operation inside a loop that re-runs unnecessarily	Same underlying performance principle: do expensive work once, cache the result, and only redo it when a genuine input changed — vs. recomputing it on every iteration regardless.
Text to Columns	`Series.str.split(delimiter, expand=True)`	Splits a whole pandas column of delimited text into several columns in one vectorised call, applying instantly to every row — Text to Columns' "breaks on new rows" limitation (§3.3) simply does not exist here, since re-running the script naturally covers however many rows the data currently has.
Column width / row height	Cosmetic `openpyxl` properties on the output file	See listing below — purely presentational, with no equivalent effect on how pandas processes the data itself.

Listing 1: Text to Columns, the pandas way — generalises automatically to any number of rows

```python
import pandas as pd

df = pd.DataFrame({"full_name": ["Sharma, Anjali", "Verma, Rohit", "Iyer, Kavya"]})

# Splits EVERY row in one call, no matter how many there are --
# unlike the Excel wizard, this never needs to be "re-run" as rows are added.
split_cols = df["full_name"].str.split(", ", expand=True)
df["last_name"], df["first_name"] = split_cols[0], split_cols[1]
print(df)
#          full_name last_name first_name
# 0   Sharma, Anjali     Sharma     Anjali
# 1     Verma, Rohit      Verma      Rohit
# 2     Iyer, Kavya       Iyer      Kavya
```

Listing 2: Cosmetic worksheet optimisation with openpyxl

```python
from openpyxl import Workbook
```

```
wb = Workbook()
ws = wb.active

# Equivalent of View -> Gridlines (unchecked)
ws.sheet_view.showGridLines = False

# Equivalent of manually setting column width / row height
ws.column_dimensions["A"].width = 18
ws.row_dimensions[1].height = 20

wb.save("clean_dashboard.xlsx")
```

SQL has no real analogue for cosmetic optimisation (a table has no font or gridlines), but its *performance*-optimisation story is directly parallel to Excel's calculation-mode discussion:

Listing 3: SQL performance optimisation: an index is the closest relative of Excel's calculation-mode discipline

```
-- Without an index, this scans every row in the table every time it runs
   --
-- the SQL equivalent of a whole-column SUM(A:A) forcing Excel to walk
-- all 1,048,576 rows on every keystroke.
SELECT * FROM monthly_expenses WHERE category = 'Rent';

-- Building an index lets the database jump straight to matching rows
-- instead of scanning the whole table -- conceptually the same payoff
-- as swapping a volatile OFFSET for a non-volatile INDEX in Excel.
CREATE INDEX idx_category ON monthly_expenses (category);
```

4 Macros and VBA: Automating Repetitive Tasks

4.1 What a Macro Actually Is

Key Concept

A **macro** is a saved, named sequence of Excel actions that can be replayed on demand — either by clicking a button, pressing an assigned shortcut, or calling it from other code. Under the hood, every macro is a small program written in **VBA** (Visual Basic for Applications), Excel's built-in programming language. You do not have to *write* VBA to create a macro: the **Macro Recorder** watches every click and keystroke you make and automatically generates the equivalent VBA code, which is how most people first encounter macros. This makes a macro conceptually identical to a short Python script that automates a repetitive task — the difference is that the "recorder" writes the first draft of the code for you by literally watching what you did, rather than you writing it from a blank file.

4.2 Enabling the Developer Tab

Macro recording and the VBA editor live on the **Developer** tab, which is hidden by default.

Enabling Developer

File → **Options** → **Customize Ribbon** → in the right-hand list of main tabs, check the box next to **Developer** → **OK**. This is a one-time, per-installation setting (§3.8).

4.3 Recording a Macro

Developer → Record Macro

1. **Macro name**: no spaces allowed (use underscores), must start with a letter.
2. **Shortcut key** (optional): assigns `Ctrl+<letter>` (or `Ctrl+Shift+<letter>`) to run the macro instantly.
3. **Store macro in**: **This Workbook** (macro travels with this file only), **New Workbook**, or **Personal Macro Workbook** (macro becomes available in *every* workbook you open — see §4.9).
4. Click **OK** — recording begins immediately. Perform the exact clicks, formatting, formula entries, or menu actions you want automated.
5. **Developer → Stop Recording** when finished.

Relative vs. Absolute References While Recording

Next to **Record Macro** is a toggle button, **Use Relative References**, and it fundamentally changes what gets recorded — this is one of the most consequential and least understood settings for anyone recording macros.

- **Off (default, absolute)**: the macro records the *exact cell addresses* you clicked — "select `A1:D4`, apply bold". Running the macro again always acts on `A1:D4`, regardless of where your cursor currently is.
- **On (relative)**: the macro records actions *relative to wherever the active cell was when recording started* — "select 4 rows down and 3 columns right from the starting cell". Running the macro again acts relative to *wherever your cursor currently is* when you run it.

Neither setting is "correct" in general — absolute is right for a macro that always formats the same fixed report header; relative is right for a macro meant to be run starting from different cells each time (e.g., "bold whatever row I've currently selected"). Recording with the wrong one selected is the single most common reason a macro that worked once behaves unexpectedly the second time.

4.4 Saving Macros: the .xlsm File Format

Macros Are Silently Discarded by the Standard .xlsx Format

A plain `.xlsx` file **cannot** contain VBA code at all. If you record a macro and then save with **Ctrl+S** in a workbook still named `.xlsx`, Excel will pop up a warning and, if you proceed, will strip every macro out of the saved file — silently, with no further prompt. You must use **File → Save As** and explicitly choose **Excel Macro-Enabled Workbook (*.xlsm)** as the file type *before* the macro is saved for good.

Extension	Can contain macros?	Notes
`.xlsx`	No	Standard format; safest to share externally since it cannot carry executable code.
`.xlsm`	Yes	Identical to `.xlsx` in every other respect; the "m" is the only signal that macros are present.
`.xlsb`	Yes (binary)	A binary (non-XML) format, faster to open/save for very large workbooks, and what the Personal Macro Workbook itself uses (`PERSONAL.XLSB`).

4.5 Macro Security

Because a macro is executable code, Excel treats macro-enabled files with deliberate caution: opening an `.xlsm` from an untrusted source (downloaded, emailed) typically opens in **Protected View** with macros disabled and a yellow **"Enable Content"** bar, which requires an explicit, informed click before any macro can run. **File → Options → Trust Center → Trust Center Settings → Macro Settings** controls the default behaviour (from "disable all macros with notification" to "enable all macros", the latter strongly discouraged). This caution exists because macros historically have been (and still are) a common vector for malware distributed as seemingly innocuous spreadsheet attachments — never enable macros in a file from a source you don't trust, and never disable this protection workbook-wide just to remove the prompt.

4.6 Reading and Editing Recorded VBA Code

Opening the VBA Editor

`Alt+F11` (or **Developer → Visual Basic**) opens the **VBA Editor**, a separate application window with its own **Project Explorer** (a tree of every open workbook and its **Modules**, where recorded macro code lives) and a code-editing pane.

Worked example. Recording a macro that bolds a header row and applies a currency number format to `B2:B6` (absolute references, recorded exactly as described in §4.3) produces VBA code that looks like this:

Listing 4: Recorded macro: format a header row and a fixed range of amounts

```
Sub FormatExpenseReport()
'
' FormatExpenseReport Macro
'

    Range("A1:B1").Select
    Selection.Font.Bold = True
    Range("B2:B6").Select
    Selection.NumberFormat = "$#,##0.00;[Red](#,##0.00)"
    Range("A1").Select
End Sub
```

Reading Recorded Code

Every line follows the same pattern the recorder always uses: **select something, then act on the Selection**. `Range("A1:B1").Select` is the VBA equivalent of clicking and dragging across those cells; `Selection.Font.Bold = True` is the equivalent of clicking the Bold button. This "select, then act" style is verbose and not how a human VBA programmer would write it from scratch (you can act on a `Range` object directly, without selecting it first), but it is exactly how the recorder works, because the recorder can only capture actions performed *on whatever is currently selected* — it has no concept of your underlying intent, only of the clicks you physically made.

4.7 The "Only Works for 4 Rows" Problem, Solved

This is the single most important limitation to understand about recorded macros, and it applies to *any* recorded action, not just Text to Columns.

Why the Recorder Always Hard-Codes a Fixed Range

Suppose you record a macro that runs **Text to Columns** on `A1:A4` because your sheet had exactly 4 rows of data at the time. The recorder writes:

Listing 5: Recorded Text to Columns – hard-coded to exactly 4 rows

```
Sub SplitNames()
    Range("A1:A4").Select
    Selection.TextToColumns Destination:=Range("A1"), DataType:=
        xlDelimited, _
         Comma:=True
End Sub
```

Run this macro next month on a sheet with 40 rows of data, and it will split *only* rows 1–4, leaving rows 5–40 completely untouched — with no error message, because as far as VBA is concerned nothing went wrong: it was told to act on `A1:A4`, and it did exactly that. The macro recorder has no way to record "and keep doing this for however many rows exist" because that intent was never expressed by any click — it can only record the literal, specific range you selected.

The fix is to manually edit the recorded code, replacing the hard-coded range with one calculated dynamically at the moment the macro runs:

Listing 6: Fixed version: dynamically finds the last used row instead of assuming 4

```
Sub SplitNames()
    Dim lastRow As Long
    ' Find the last row that actually contains data in column A,
    ' whatever that row number happens to be today.
    lastRow = Cells(Rows.Count, "A").End(xlUp).Row

    Range("A1:A" & lastRow).Select
    Selection.TextToColumns Destination:=Range("A1"), DataType:=
        xlDelimited, _
         Comma:=True
End Sub
```

The Pattern to Memorise

`Cells(Rows.Count, "A").End(xlUp).Row` is the VBA idiom for "start at the very bottom of column A and jump up to the first cell that actually has data in it" — it is the code equivalent of pressing `Ctrl+↑` from the bottom of the sheet. This single line, substituted for any hard-coded row number a recorder wrote, is what turns a brittle "works for exactly the rows I had today" macro into one that correctly generalises to however many rows exist whenever it is next run. The same idiom, swapping `"A"` for the relevant column, fixes the identical problem for any recorded macro (formatting, sorting, applying a formula down a column) that was recorded against a fixed number of rows.

4.8 Writing a Macro From Scratch: Basic VBA Building Blocks

Recording is a fast way to get a first draft, but real fluency comes from being able to read and write a few VBA basics directly, the same way you'd write a short script instead of always relying on a UI recorder.

Construct	Meaning
`Sub Name() ...End Sub`	Defines a macro (a procedure with no return value) — the VBA equivalent of `def name():` in Python.
`Dim x As Long`	Declares a variable `x` with an explicit type (`Long` = integer, `String`, `Double` = decimal, `Range`, `Boolean`, ...) — VBA requires (or at least strongly encourages, via `Option Explicit`) declared types, unlike Python's dynamic typing.
`Range("A1").Value`	Reads or writes the value in a specific cell — equivalent of `ws["A1"].value` in `openpyxl`.
`Cells(r, c)`	Refers to a cell by numeric row/column index, e.g. `Cells(2, 1)` = `A2` — convenient inside loops where row/column numbers are variables.
`For i = 1 To n ...Next i`	A counted loop, directly analogous to Python's `for i in range(1, n+1):`.
`If ...Then ...Else ...End If`	Conditional branching, identical in spirit to Python's `if/else`.
`MsgBox "text"`	Pops up a message dialog — useful for confirmations or simple debugging, analogous to `print()` but as a UI dialog rather than console output.

Listing 7: A macro written from scratch: highlight every negative value in a used range, red text

```
Sub HighlightNegatives()
    Dim lastRow As Long, i As Long

    ' Dynamic range from the start -- written this way deliberately,
    ' not recorded, so it never has the "only 4 rows" problem at all.
    lastRow = Cells(Rows.Count, "B").End(xlUp).Row

    For i = 2 To lastRow                    ' skip row 1, the header
        If Cells(i, "B").Value < 0 Then
            Cells(i, "B").Font.Color = RGB(255, 0, 0)
        Else
            Cells(i, "B").Font.Color = RGB(0, 0, 0)
        End If
    Next i
```

```
15
16      MsgBox "Formatted " & (lastRow - 1) & " rows."
17  End Sub
```

Finance Application

A version of exactly this pattern — loop down a column, test a condition, apply formatting or flag a value — is how many finance teams automate month-end tasks before a report goes out: flagging variances beyond a tolerance, colour-coding overdue receivables by age, or marking any expense line that breaches its budget. Module 3's **Conditional Formatting** achieves a similar visual result natively without VBA for simple rules; a macro becomes worthwhile once the logic is more involved than Conditional Formatting's rule engine can express, or needs to trigger a side effect (a `MsgBox` alert, writing a log entry) rather than just changing appearance.

4.9 The Personal Macro Workbook

Choosing **Personal Macro Workbook** as the storage location when recording (§4.3) saves the macro into a special hidden workbook, `PERSONAL.XLSB`, stored inside the **XLSTART** folder introduced in §3.9. Because everything in `XLSTART` opens automatically with Excel, `PERSONAL.XLSB` — and every macro inside it — becomes available from *any* workbook you have open, not just the one it was recorded in. This is the standard way to build a personal library of general-purpose utility macros (a formatting cleanup routine, a standard report layout) that should follow you across every file you work on, as opposed to a macro that only makes sense embedded in one specific model.

4.10 Running a Macro: Buttons, Shapes, and the Quick Access Toolbar

A macro assigned only to a typed shortcut key is invisible to anyone who doesn't already know it exists — for a workbook shared with less technical colleagues, macros are normally exposed as a clickable control:

- **Insert** → **Shapes** (or a rectangle drawn as a "button"), then right-click it → **Assign Macro** → pick the macro from the list.
- **Developer** → **Insert** → **Form Controls** → **Button**, which inserts a native, click-styled button object and immediately prompts for a macro to assign.
- Adding to the **Quick Access Toolbar** (right-click the QAT → **Customize Quick Access Toolbar** → choose "Macros" from the command list) for a macro you personally run constantly across many workbooks.

4.11 The Real Limitations of Macros

Understanding what macros are *not* good at is as important as knowing how to build them, and is exactly why a tech-background analyst will often reach for Python instead once automation needs grow past a certain point.

What Macros Cannot Do Gracefully

- **They record literal actions, not intent.** As shown in §4.7, a recorder captures exactly what you clicked, never the general rule you had in mind — every recorded macro needs a human to manually generalise it before it is safe to reuse.
- **They are brittle to layout changes.** A macro referencing `Range("B2:B6")` breaks silently (or acts on the wrong cells) the moment a row is inserted above it, a column is deleted, or a sheet is renamed — there is no equivalent of a compiler catching this before it causes a wrong result.
- **No built-in error handling by default.** Unlike Python's `try/except`, a bare VBA macro simply halts with an opaque runtime error dialog the moment something unexpected happens (a cell contains text where a number was expected, a sheet doesn't exist) — explicit `On Error` handling has to be added deliberately.
- **Weak outside Excel itself.** Macros can drive other Office applications and, with effort, make web requests or read files, but this is clunky, poorly documented compared to Python's ecosystem, and rarely how it's actually done in practice — there is no VBA equivalent of `pip install requests` or `pandas`.
- **No real version control story.** VBA code lives *inside* the binary/XML workbook file itself; diffing what changed in a macro between two versions of a workbook, or code-reviewing a change, is far clumsier than reviewing a `.py` file's history in Git.

When to Reach for Macros vs. Python

Macros remain the right tool when the automation is **simple, Excel-native, and needs to travel inside the workbook itself** for a non-technical end user to trigger with one click — formatting cleanup, a standard print setup, a repeated report layout. Python (typically via `openpyxl`, or driving Excel directly via `xlwings`/`pywin32`) becomes the better tool once the task involves substantial data transformation, needs to run unattended on a schedule, needs proper error handling and logging, needs to integrate with something outside Excel (a database, an API, another file format), or simply needs to be maintained and version-controlled like real code. In many finance teams, both coexist deliberately: a Python pipeline builds and refreshes the underlying data, and a light macro or two polishes the final presentation layer inside the delivered workbook.

4.12 Cross-Tool Translation: Python and SQL

Listing 8: The same "highlight negatives" macro from S3.8, in Python – notice it needs no dynamic-range fix, because it naturally operates on however many rows exist

```python
from openpyxl import load_workbook
from openpyxl.styles import Font

wb = load_workbook("expenses.xlsx")
ws = wb["Monthly Expenses"]

# ws.max_row always reflects the CURRENT sheet, so this script
# never has the VBA "hard-coded to 4 rows" problem in the first place --
# the loop bound is recomputed fresh every time the script runs.
for row in range(2, ws.max_row + 1):
    cell = ws.cell(row=row, column=2)    # column B
    if isinstance(cell.value, (int, float)) and cell.value < 0:
        cell.font = Font(color="FF0000")
    else:
        cell.font = Font(color="000000")

wb.save("expenses.xlsx")
```

```
print(f"Formatted {ws.max_row - 1} rows.")
```

Why Python Sidesteps the "Only 4 Rows" Problem by Default

The VBA fix in §4.7 required deliberately adding `Cells(Rows.Count,"A").End(xlUp).Row` to escape a hard-coded range — an extra step you have to remember. In Python, computing a bound like `ws.max_row` (or, in pandas, simply operating on `df` however many rows it currently holds) is the *natural*, default way of writing the loop in the first place; producing a script that only ever processes exactly 4 rows would actually take deliberate, unusual extra effort. This is a genuine structural advantage of a code-first tool over a click-recorder: intent ("every row") is what you write from the start, rather than something you retrofit after discovering the recorder only captured 4 specific rows.

Listing 9: xlwings: driving live Excel (and even calling VBA) from Python, when you need Excel's own engine rather than just editing the file

```
# pip install xlwings
import xlwings as xw

app = xw.App(visible=False)
wb = app.books.open("expenses.xlsx")
sht = wb.sheets["Monthly Expenses"]

sht.range("B2").value = 25000           # writes a live cell value
result = sht.range("B7").value          # reads back a formula's
    calculated result

wb.macro("FormatExpenseReport")()       # can even call an existing VBA
    macro directly

wb.save()
app.quit()
```

Finance Application

`xlwings` (and, on Windows, the lower-level `pywin32`) is the bridge used when a workflow genuinely needs Excel's own calculation engine running — for example, driving a legacy, VBA-heavy pricing model from an automated Python pipeline without rewriting the model's formulas, or triggering Excel to open, refresh, and re-save hundreds of report files on a schedule.

SQL's closest conceptual relative to a macro is a **stored procedure**: a named, saved sequence of SQL statements that can be invoked repeatedly instead of retyped.

Listing 10: A stored procedure – SQL's version of "a saved, repeatable, named sequence of actions"

```
CREATE PROCEDURE FlagOverBudgetExpenses()
BEGIN
    UPDATE monthly_expenses
    SET is_flagged = TRUE
    WHERE amount > 20000.00;
END;

-- "Running the macro":
CALL FlagOverBudgetExpenses();
```

Key Concept

The parallel is deliberately close: both a VBA macro and a SQL stored procedure package a repeatable action under one name, both can be parameterised, and both can be triggered by an event rather than a manual call (VBA's `Auto_Open`, SQL's `TRIGGER`). The difference in practice is scope — a macro typically manipulates the visual, formatted surface of a workbook, while a stored procedure manipulates the underlying data of a table, with no concept of "formatting" at all.

5 Absolute vs. Relative Cell References

5.1 Recap: Why This Matters

Module 1 flagged this exact topic and deliberately deferred it:

From Module 1

By default every reference you type is a **relative reference**: copying a formula shifts every reference by the same offset as the copy. This is exactly what you want when copying a formula down a column of similar rows, and exactly what you don't want when a formula must always point at one fixed cell — such as a tax-rate assumption referenced by every row in a model.

This section makes that distinction fully precise, because it is one of the highest-frequency sources of silent, hard-to-spot errors in real financial models — a formula that produces a *plausible-looking but wrong* number after being copied is far more dangerous than one that produces an obvious `#REF!` error.

5.2 The $ Symbol: Locking a Reference

A `$` placed immediately before a column letter or row number **locks** that part of the reference, so it does not shift when the formula is copied elsewhere. The column and row locks are entirely independent of each other.

The Four Reference Types

Reference	Type	Behaviour when copied
`A1`	Relative	Both column and row shift with the copy.
`$A$1`	Absolute	Neither column nor row shifts — always points at `A1`, no matter where the formula is copied.
`$A1`	Mixed (column locked)	Column stays fixed at `A`; row shifts normally.
`A$1`	Mixed (row locked)	Row stays fixed at `1`; column shifts normally.

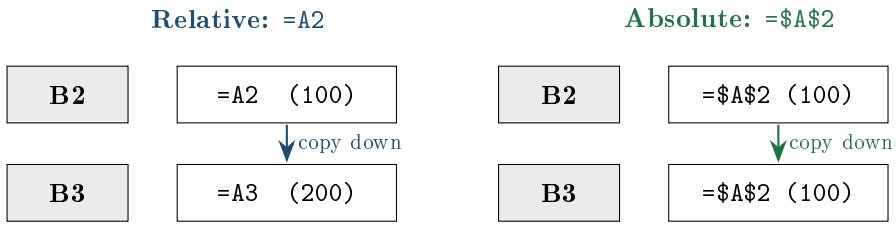


Figure 1: Copying a formula one row down. The relative reference `A2` silently becomes `A3`, tracking the source data correctly in this case; the absolute reference `$A$2` deliberately refuses to move, always returning to the same fixed cell regardless of where the formula lives.

5.3 The `F4` Cycling Shortcut

F4: Cycle Through Reference Types Without Retyping

With the cursor positioned inside or immediately after a cell reference in the formula bar, pressing `F4` repeatedly cycles through all four states in a fixed order:

`A1` → `$A$1` → `A$1` → `$A1` → `A1` → ···

This is dramatically faster than manually typing `$` symbols, especially inside a long formula with several references to lock, and is one of the highest-value habits to build early — fluent Excel users use `F4` reflexively rather than typing dollar signs by hand.

5.4 Mixed References in Practice: A Two-Way Sensitivity Grid

Mixed references exist for exactly one recurring scenario: a grid where **one axis should stay locked to a row, and the other axis should stay locked to a column** — the classic use case being a sensitivity table with one assumption varying across the top and a second assumption varying down the side.

Revenue Sensitivity Grid

Row 1 (`B1:E1`) holds four **base revenue** assumptions; column A (`A2:A5`) holds four **growth rate** assumptions. A single formula, typed once into `B2` and then copied across the *entire* grid `B2:E5`, should correctly compute *every* combination:

The Grid Formula — Typed Once Into B2

```
=B$1*(1+$A2)
```

- `B$1` — column **unlocked** (so it tracks whichever revenue column the formula currently sits under), row **locked** at `1` (so it always reads the header row, never a row of results).
- `$A2` — column **locked** at `A` (so it always reads the growth-rate column, never drifts right into the results), row **unlocked** (so it tracks whichever growth-rate row the formula currently sits in).

	B: 100	C: 150	D: 200	E: 250
Row 2: 5%	105.0	157.5	210.0	262.5
Row 3: 10%	110.0	165.0	220.0	275.0
Row 4: 15%	115.0	172.5	230.0	287.5
Row 5: 20%	120.0	180.0	240.0	300.0

Key Concept

Every one of these 16 cells is the *identical* formula text `=B$1*(1+$A2)` typed exactly once, then filled right and down — the mixed locks do all the work of making the column reference track across and the row reference track down independently. Getting even one lock wrong (e.g., typing `$B$1` fully absolute) would make every cell in the grid read the same single revenue figure instead of tracking its own column, producing a grid that looks plausible at a glance but is silently wrong everywhere except column B.

Excel's Native "Data Table" Feature

What-If Analysis → Data Table (Data tab) builds exactly this kind of two-way grid natively, without you writing the mixed-reference formula by hand — you provide one formula referencing your model's live inputs, plus a row of column-input values and a column of row-input values, and Excel fills the grid by silently re-running the whole calculation once per cell. This is convenient, but recall the performance pitfall from §3.10: Data Tables recalculate on *every* recalculation event by default and can become the single most expensive object in a model, which is precisely why **Automatic Except for Data Tables** exists as a calculation-mode option.

5.5 References Across Sheets and Workbooks

The same `$` locking applies identically to references that cross a sheet or workbook boundary:

Cross-Sheet and Cross-Workbook References

- Same workbook, another sheet: `='Assumptions'!$B$4` — the sheet name is followed by `!`, and is wrapped in single quotes only if it contains a space or starts with a number.
- Another, currently open workbook: `='[Model.xlsx]Assumptions'!$B$4`.
- Another, closed workbook: identical syntax, but Excel additionally embeds the full file path, and the formula returns a cached last-known value until that workbook is next opened and refreshed.

Revisiting the Tax-Rate Scenario

Module 1 previewed this exact scenario: a tax-rate assumption sits in one cell, referenced by a formula in every row of a P&L. Written with a relative reference (`=B10*TaxCell`) and copied down 50 rows, the reference to `TaxCell` silently drifts down 50 rows too — by row 30 it may be reading a blank cell (silently returning `0`, no error at all) or, worse, a completely unrelated number that happens to sit in that position. The fix is either an absolute reference (`=B10*$B$4`) or, cleaner still, a **Named Range** (`=B10*TaxRate`) — the subject of the next section, which solves this problem even more robustly by removing the raw cell address from the formula altogether.

5.6 Cross-Tool Translation: Python and SQL

Why This Problem Is Almost Unique to Excel

Python and SQL do not really have an equivalent bug class here, and it's worth understanding precisely why. The entire relative/absolute distinction exists because Excel's mental model is "write one formula, then *copy* it many times, and let each copy silently adjust." Python's mental model is the opposite: you write *one* operation and apply it to an entire column or table at once (**vectorisation**), so there is no "copy" step during which a reference could drift. A pandas operation like `df["Tax"] = df["Revenue"] * tax_rate` runs once, references `tax_rate` exactly once, and applies to every row simultaneously — there is no analogue of accidentally letting `tax_rate` drift to a different value on row 30, because there was never a per-row copy of the formula in the first place.

Listing 11: The "always reference one fixed value" pattern, done the Python way – a scalar constant, referenced once, broadcast automatically

```python
import pandas as pd

df = pd.DataFrame({"Revenue": [100_000, 150_000, 200_000]})

TAX_RATE = 0.21    # a single, unambiguous, named constant -- the Python
                   # equivalent of what a Named Range achieves in Excel

# No "copying a formula down 50 rows" step exists here, so there is no
# reference to accidentally let drift -- pandas broadcasts the scalar
# across every row of the column in one call.
df["Tax"] = df["Revenue"] * TAX_RATE
print(df)
#    Revenue       Tax
# 0   100000   21000.0
# 1   150000   31500.0
# 2   200000   42000.0
```

SQL's story is similar, but worth stating precisely: a SQL table also has no concept of a copied formula "drifting", because a column's computed value (whether stored or computed on the fly in a query) is defined once, by name, and applies uniformly to every row that query touches.

Listing 12: The same fixed-rate calculation in SQL – a literal constant applied uniformly, or joined from a single-row config table

```sql
-- Option A: literal constant in the query itself
SELECT
    revenue,
    revenue * 0.21 AS tax
FROM company_financials;

-- Option B: a single-row "assumptions" table, joined in --
-- the SQL analogue of referencing a Named Range instead of
-- hard-coding the number, so the rate lives in exactly one place
SELECT
    f.revenue,
    f.revenue * a.tax_rate AS tax
FROM company_financials AS f
CROSS JOIN assumptions AS a;   -- assumptions has exactly one row
```

Key Concept

The `CROSS JOIN` against a deliberately single-row `assumptions` table is the closest SQL gets to the Excel pattern of "one cell that every row's formula reaches back to" — every output row is paired with the same one assumptions row, the same way every cell in a column referencing `$B$4` reaches back to the same one input cell. Both approaches share the same benefit over hard-coding the number directly: change the rate in exactly one place (the config table, or the referenced cell), and every dependent row updates correctly and simultaneously.

6 Named Ranges

6.1 Why Named Ranges Exist

From Module 1 — Now Fully Resolved

Module 1 identified the core weakness of Excel's addressing system: a cell address like `C18` is **positional** — "third column, eighteenth row" — not **semantic**. It says nothing about what the cell actually represents. A **Named Range** lets you attach a meaningful, stable name to a cell or range, so a formula can say `=Revenue - COGS` instead of `=B10-B11`. This is the single biggest step toward making a formula *self-documenting*: a reader (including you, six months later) can understand what a formula computes without first clicking every cell it references to discover what each one holds.

6.2 Creating a Named Range

Three Ways to Create a Name

1. **Name Box** (the small box at the far left of the formula bar, normally showing the active cell's address): select the range first, then click into the Name Box, type the name, and press `Enter`. The fastest method for a single, quick name.
2. **Formulas → Define Name**: opens a dialog with explicit fields for Name, Scope, Comment, and the range itself (**Refers to**) — preferred when you want to set the scope deliberately (§6.5) or add a comment.
3. **Formulas → Create from Selection**: select a range *together with* its adjacent header row/column, and Excel automatically creates one name per data column (or row), using the header text as the name — by far the fastest way to name an entire table's columns in one action.

6.3 Naming Rules

Rule	Detail
Must start with a letter or underscore	Cannot start with a number or most symbols.
No spaces	Use an underscore or CamelCase instead: `Tax_Rate` or `TaxRate`, never `Tax Rate`.
Cannot look like a cell address	`Q3` is rejected, since it is indistinguishable from the cell address `Q3`; `Q3_Revenue` is fine.
Case-insensitive	`TaxRate` and `taxrate` refer to the *same* name — Excel does not allow both to exist as distinct names simultaneously.
Reasonable length limit	Up to 255 characters, though names are conventionally kept short and readable.

6.4 Using a Named Range in a Formula

Once defined, a name can be typed directly into any formula exactly like a cell reference, and — critically — **behaves as an absolute reference by default**: typing `TaxRate` into a formula and copying that formula anywhere else always resolves back to the same named cell, with no `$` signs required at all.

Listing 13: Before and after naming a tax-rate assumption cell

```
Before:  =B10*$B$4
After:   =B10*TaxRate
```

Finance Application

Named ranges are also usable across sheets without repeating the sheet-name prefix: if `TaxRate` is defined at workbook scope (§6.5) on an "Assumptions" tab, a formula on a completely different "Income Statement" tab can simply write `=Revenue*TaxRate`, with no `'Assumptions'!` prefix needed anywhere — the name itself already knows where it points.

6.5 Scope: Workbook vs. Worksheet

Every name has a **scope**, set at creation time in the Define Name dialog:

- **Workbook scope** (the default): the name is valid and resolves to the same cell from *any* sheet in the workbook. Almost always what you want for a global assumption like a tax rate or a reporting currency.
- **Worksheet scope**: the name is only valid *on that one sheet* — using it from another sheet either produces an error or, more dangerously, silently refers to a *different* name with the same text if one happens to exist at workbook scope or on the current sheet.

Scope Collisions

A worksheet-scoped name can legally share the same text as a different worksheet-scoped name on another sheet — for example, both a "Q1" tab and a "Q2" tab defining their own local name `StartDate`, each pointing at a different cell. This is occasionally useful (a template pattern repeated identically per tab) but is also a common source of confusion: a formula copied from the Q1 tab to the Q2 tab that references `StartDate` will silently resolve to the *Q2* tab's own `StartDate`, not the one it originally pointed to, with no warning. When in doubt, prefer workbook-scoped names and give them specific, unambiguous text (`Q1_StartDate`, `Q2_StartDate`) instead.

6.6 The Name Manager

Formulas → Name Manager (or `Ctrl+F3`)

Opens a single dialog listing every defined name in the workbook, its scope, its current **Refers to** range, and any comment — with buttons to **Edit** (redefine what a name points to, without touching any formula that uses it), **Delete**, or **Filter** the list (e.g. to show only names with errors). This is the single place to audit every name in a large model at once, and the first place to check when a formula referencing a name returns an unexpected result.

6.7 Named Ranges as Data Validation List Sources

One of the most common practical uses of a named range is supplying the source list for a Data Validation dropdown (§7) — rather than typing a literal comma-separated list into the Data Validation

dialog itself, you point it at a named range, so updating the named range's contents automatically updates every dropdown built from it, everywhere in the workbook.

6.8 Dynamic Named Ranges and the Modern Alternative: Excel Tables

A named range created via the Name Box or Define Name points to a *fixed* address, such as `Assumptions!$A$2:$A$10`. If new rows are added below row 10, the name does not automatically grow to include them.

The Classic Fix — OFFSET/COUNTA — and Why It's Rarely Used Today

Older Excel guidance defines a name whose **Refers to** field uses `OFFSET` combined with `COUNTA` to calculate a range's size dynamically, e.g. `=OFFSET(Assumptions!$A$2,0,0,COUNTA(Assumptions!$A:$A)-1,1)`, which grows or shrinks automatically as data is added or removed. This works, but `OFFSET` is **volatile** (§3.10) — it forces a recalculation on every change anywhere in the workbook — so this classic pattern has a real, measurable performance cost in a large model.

The Modern Answer: Excel Tables

Selecting a data range and pressing `Ctrl+T` (or **Insert** → **Table**) converts it into an **Excel Table** — a structured object that automatically expands its range as new rows are typed directly below it, carries its own **structured references** (`Table1[Revenue]` instead of `$B$2:$B$47`), and is *not* volatile. A structured reference to a Table column is, for almost every practical purpose, a self-expanding named range without the `OFFSET` performance penalty, and is the modern, recommended replacement for dynamic named ranges in current Excel. It is introduced here because it depends conceptually on named ranges to understand, but a full treatment of Tables, structured references, and their interaction with lookup functions belongs alongside Module 3's lookup-function coverage.

6.9 Named Constants: a Name With No Cell At All

A name does not have to refer to a range — the **Refers to** field in Define Name can instead hold a literal value or formula directly, with no cell reference at all:

Listing 14: Define Name dialog: Name = TaxRate, Refers to = 0.21 (no cell involved)

```
Name:        TaxRate
Refers to:   =0.21
```

Key Concept

This is the Excel equivalent of a named constant in code (`TAX_RATE = 0.21` in Python) — useful for a value that is genuinely fixed and unlikely to need its own dedicated input cell (a rounding tolerance, a fiscal year-end month number), though most finance teams still prefer a visible input *cell* for any assumption a user might reasonably want to change or audit, reserving named constants for true structural constants.

6.10 Common Pitfalls

#REF! From Deleting a Named Range's Source

Deleting the row or column a named range points to does **not** delete the name itself — the name survives in the Name Manager but its **Refers to** field becomes `#REF!`, and every formula using that name breaks simultaneously across the entire workbook. Always check the Name Manager before deleting rows/columns in a model that uses named ranges, and be aware that a single deletion can silently break formulas on sheets you weren't even looking at.

Names Don't Follow a Copied Sheet the Way You'd Expect

Copying an entire worksheet (right-click the sheet tab → **Move or Copy** → check **Create a copy**) duplicates any worksheet-scoped names defined on it, but workbook-scoped names are *not* duplicated — formulas on the copied sheet that reference a workbook-scoped name continue pointing back at the *original* sheet's cell, not a new copy on the duplicated sheet. This is a frequent, confusing surprise when duplicating a monthly-report template.

6.11 Cross-Tool Translation: Python and SQL

Listing 15: Named ranges, the openpyxl way – defined names actually written into the .xlsx file

```python
from openpyxl import Workbook
from openpyxl.workbook.defined_name import DefinedName

wb = Workbook()
ws = wb.active
ws["A1"] = "TaxRate"
ws["B1"] = 0.21

# Registers a workbook-scoped name, exactly like Formulas -> Define Name --
# Excel itself will show "TaxRate" as a usable name when this file is opened.
wb.defined_names["TaxRate"] = DefinedName("TaxRate", attr_text="Sheet!$B$1")

ws["B2"] = "=B1*100000*TaxRate"    # a live formula string using the named range
wb.save("model.xlsx")
```

Key Concept

In plain Python (outside an actual `.xlsx` file), the equivalent of a named range is simply a well-named **variable** — `tax_rate = 0.21` instead of a magic, unexplained `0.21` scattered through the code. This is such a basic, automatic habit in programming that it is easy to forget it is exactly the same discipline Excel needs deliberate effort (defining a name, remembering to use it instead of the raw cell) to achieve — code enforces readable references by convention; Excel requires you to opt in.

SQL's closest equivalents are a **view** (a saved, named, reusable query definition) and a **CTE** (Common Table Expression, a name scoped to one query):

Listing 16: A view: a persistent, named definition usable in any future query, like a workbook-scoped named range

```sql
CREATE VIEW high_value_expenses AS
```

```
    SELECT * FROM monthly_expenses WHERE amount > 20000.00;

-- Used later, by name, exactly like referencing TaxRate in a formula:
SELECT * FROM high_value_expenses WHERE category = 'Rent';
```

Listing 17: A CTE: a name scoped only to the one query it's defined in, like a worksheet-scoped named range

```
WITH recent_expenses AS (
    SELECT * FROM monthly_expenses WHERE expense_date >= '2026-07-01'
)
SELECT category, SUM(amount)
FROM recent_expenses
GROUP BY category;
-- "recent_expenses" does not exist outside this one query,
-- just as a worksheet-scoped name does not exist outside its own sheet.
```

7 Data Validation and Custom Validation Formulas

7.1 Why Data Validation Exists

Key Concept

A blank Excel cell will accept absolutely anything typed into it — a number where a date was expected, a negative value for a quantity that can never be negative, a free-text category that should have come from a fixed list of ten approved values. **Data Validation** (**Data** tab → **Data Validation**) lets you constrain, in advance, what a user is allowed to enter into a specific cell or range — Excel's closest equivalent to a database `CHECK` constraint, and the only real defense a worksheet has against bad input at the point of entry, rather than discovering the bad data later when a formula downstream produces a wrong or broken result.

7.2 The Data Validation Dialog: Three Tabs

Data → Data Validation

The dialog has three tabs, and a complete, professional validation rule normally uses all three together:

- **Settings** — defines the actual rule (what is and isn't allowed).
- **Input Message** — an optional tooltip shown *before* the user types anything, while the cell is simply selected.
- **Error Alert** — what happens *after* an invalid entry is attempted.

7.3 The Settings Tab: Allow Types

Allow	Extra options	Typical use
Any value	—	The default (no restriction) — effectively "no validation".
Whole number	between / not between / equal to / greater than / less than (...) a value or cell reference	Headcount, number of units, a year entered as `2026` rather than a date.
Decimal	Same comparison operators	A growth rate, a margin, any bounded numeric assumption.
List	A literal comma-separated list, *or* a cell range/named range	The single most common rule in finance models: forcing a category, scenario name, or account type to come from a fixed, approved set (§7.4).
Date	between / not between / ..., comparing to a fixed date or another cell	A transaction date that must fall within the current fiscal period.
Time	Same comparison operators, for time-of-day values	Less common in finance; more typical in operational/scheduling sheets.
Text length	between / not between / ..., comparing character counts	Enforcing a fixed-length code (e.g. exactly 10-digit account numbers).
Custom	Any formula that evaluates to `TRUE` or `FALSE`	Any rule the built-in types can't express directly (§7.6) — the most powerful and most under-used option.

7.4 List Validation: Building a Dropdown

List Validation — Two Sources

1. **Literal list**: **Allow:** **List** → **Source**: type values directly, separated by commas, e.g. `Best Case,Base Case,Worst Case`. Quick, but the list is buried inside the dialog and must be edited there directly if it ever changes.
2. **Range or Named Range source**: **Source**: reference a range, e.g. `=$H$2:$H$4`, or better, a **Named Range** (§6), e.g. `=ScenarioList`. Editing the named range's contents automatically updates *every* dropdown built from it, everywhere in the workbook — the clear best practice for any list likely to change or reused in more than one place.

Checking **In-cell dropdown** (on by default) adds the small dropdown arrow that appears when the cell is selected; unchecking it still enforces the rule but with no visible arrow, occasionally used when a dropdown arrow would clutter a dense input area.

Scenario Switches

A dropdown list of `Best Case / Base Case / Worst Case`, sourced from a named range, feeding into an `INDEX/MATCH` or `XLOOKUP` formula (both covered in Module 3) that pulls the corresponding set of assumptions, is the standard mechanism behind an interactive "scenario switch" in a financial model — letting a user flip the entire model's output between cases with one dropdown selection instead of manually overtyping a dozen assumption cells.

Preventing Typos That Silently Break Aggregation

A free-text "Category" column where one row is typed `"Rent"` and another is typed `"rent "` (trailing space) or `"Rentals"` will cause a `COUNTIF/SUMIF` (Module 3) grouping by category to silently split what should be one category into two or three, understating totals with no error message. Constraining that column to a List validation sourced from an approved category list eliminates this entire class of bug at the point of entry, rather than requiring after-the-fact data cleaning.

7.5 Whole Number, Decimal, Date, and Text Length Validation

Each of these follows the same pattern: choose a comparison operator, then supply either literal values or — more powerfully — **cell references**, so the bound itself can be a live assumption rather than a hard-coded number.

Listing 18: A Decimal validation rule referencing a live cell as its upper bound, instead of a hard-coded number

```
Allow:      Decimal
Data:       between
Minimum:    0
Maximum:    =MaxGrowthRate      <- a Named Range, not a typed literal
```

Finance Application

Bounding a growth-rate input cell to, say, `0` to `50%` prevents an obvious fat-finger entry (typing `500` meaning `5%`, forgetting the cell is already formatted as a percentage) from silently flowing through an entire model and producing a wildly implausible — but not obviously broken — output.

7.6 Custom Validation: Writing Your Own Rule

Allow: Custom accepts any formula that evaluates to `TRUE` (entry allowed) or `FALSE` (entry rejected), evaluated as if it were typed into the *first* cell of the validated range — this is the closest Data Validation gets to writing arbitrary logic, and is what makes it genuinely powerful rather than just a set of preset dropdowns.

Goal	Custom formula (typed for the top-left cell of the range)
Prevent duplicate entries in a column	`=COUNTIF($A$2:$A$100,A2)=1`
Only allow entry if another cell isn't blank	`=NOT(ISBLANK($B2))`
Restrict text to uppercase letters only	`=EXACT(A2,UPPER(A2))`
Cap a total column at a budget ceiling	`=SUM($C$2:$C2)<=$BudgetCap`
Require text starting with a fixed prefix	`=LEFT(A2,3)="INV"`

Why the Reference Style Matters Here

Custom validation formulas rely on exactly the relative/absolute reference distinction from §5: notice `A2` (relative, so it correctly re-checks *each row's own value* as the rule is applied down a column) against `$A$2:$A$100` (absolute, so the comparison range itself never shifts). Getting this wrong — for instance, accidentally making the whole range relative too — silently produces a duplicate-check that compares each row only to itself, which always evaluates `TRUE` and enforces nothing at all.

7.7 Input Message and Error Alert

Input Message Tab

Title and **Input message** fields define a small tooltip that appears automatically whenever the cell is selected, *before* anything is typed — used to guide the user ("Enter a value between 0% and 50%") rather than to police the entry after the fact.

Error Alert Tab

Style determines what happens on an invalid entry:

- **Stop** — rejects the entry outright; the user cannot leave the cell with invalid data in it. The strictest and most common choice for hard constraints.
- **Warning** — shows a warning but lets the user override it and keep the invalid entry anyway ("Yes" / "No" / "Cancel"). Useful when a rule is a strong suggestion rather than an absolute requirement.
- **Information** — simply informs the user, with no blocking behaviour at all; the entry is always accepted regardless.

Title and **Error message** customise the dialog text shown — always worth writing a specific, helpful message ("Growth rate must be between 0% and 50%") rather than leaving Excel's generic default, which merely says the value violates the rule defined for the cell without explaining what that rule actually is.

7.8 Circling Invalid Data

Auditing Data That Was Already There

Data Validation rules only intercept *new* typed or pasted entries by default — they do **not** retroactively check data that was already sitting in the cells before the rule was applied (a very common situation when validation is added to a sheet *after* data has been pasted in from an external source). **Data → Data Validation → dropdown arrow next to Data Validation → Circle Invalid Data** draws a red circle around every existing cell in the workbook that currently violates its own validation rule, making it possible to audit and clean up pre-existing bad data in one pass rather than hunting for it manually.

7.9 Copying, Extending, and Removing Validation

- Data Validation is a cell *property*, so it copies along with **Copy/Paste** (and with **Paste Special → Validation**, which pastes *only* the rule, leaving the destination cells' existing values and formatting untouched).
- Applying a rule to an entire column (e.g. `C:C`) rather than just the currently-populated rows future-proofs the sheet against new rows added later — at the cost of a very slightly larger workbook file, a trade-off almost always worth making for an input column.
- **Data → Data Validation → Clear All** removes the rule (and, separately, Input Message/Error Alert text) from the selected cells entirely.

7.10 Cross-Tool Translation: Python and SQL

Listing 19: Data Validation, the openpyxl way – rules written directly into the .xlsx so Excel itself enforces them on open

```python
from openpyxl import Workbook
from openpyxl.worksheet.datavalidation import DataValidation

wb = Workbook()
ws = wb.active

# List validation, equivalent to Allow: List, Source: a literal list
dv_list = DataValidation(
    type="list",
    formula1='"Best Case,Base Case,Worst Case"',
    allow_blank=False,
    showErrorMessage=True,
    errorTitle="Invalid Scenario",
    error="Choose Best Case, Base Case, or Worst Case.",
)
ws.add_data_validation(dv_list)
dv_list.add("B2")

# Decimal validation, equivalent to Allow: Decimal, between 0 and 0.5
dv_decimal = DataValidation(
    type="decimal", operator="between", formula1="0", formula2="0.5",
    showErrorMessage=True, errorTitle="Out of Range",
    error="Growth rate must be between 0% and 50%.",
)
ws.add_data_validation(dv_decimal)
dv_decimal.add("B3")

wb.save("validated_model.xlsx")
```

A Genuinely Different Philosophy: Prevent vs. Detect

This is worth pausing on, because it is a real design-philosophy difference, not just a syntax difference. Excel's Data Validation is **preventative** — it stops bad data from ever being entered into the cell in the first place. The default pandas/Python approach is usually **detective** — data is loaded first, and *then* checked, typically with an `assert` statement or a dedicated validation library:

Listing 20: The pandas-native approach: validate after loading, not while typing

```
import pandas as pd

df = pd.read_csv("scenario_inputs.csv")

# Detective, not preventative -- this runs AFTER the data already
# exists in memory, and simply flags/raises if something is wrong.
assert df["growth_rate"].between(0, 0.5).all(), "Growth rate out of
    bounds!"
assert df["scenario"].isin(["Best Case", "Base Case", "Worst Case"]).all
    (), \
     "Unrecognised scenario value found!"
```

Listing 21: A schema-based library (pandera) makes the same detective checks more declarative and reusable

```
# pip install pandera
import pandera as pa
from pandera import Column, Check

schema = pa.DataFrameSchema({
    "growth_rate": Column(float, Check.in_range(0, 0.5)),
    "scenario": Column(str, Check.isin(["Best Case", "Base Case", "Worst
        Case"])),
})

schema.validate(df)    # raises a detailed SchemaError if any row violates
     a rule
```

Choosing Based on Who Is Typing

Excel's preventative style is the right choice whenever a *human* is typing data directly into cells — catching a mistake at the moment of entry is far more useful to that person than discovering it downstream. Python's detective style is the right choice for data arriving in bulk from a file, an API, or a database query, where there is no single "moment of typing" to intercept, and checking the whole batch at once after loading is both simpler and more thorough.

SQL enforces constraints preventatively too, much closer in spirit to Excel's approach than pandas' after-the-fact checks:

Listing 22: SQL constraints – the direct equivalents of each Data Validation type

```
CREATE TABLE scenario_inputs (
    scenario      VARCHAR(20) NOT NULL
                  CHECK (scenario IN ('Best Case','Base Case','Worst Case'
                      )),
                  -- equivalent of Allow: List

    growth_rate   DECIMAL(5,4) NOT NULL
```

```
                    CHECK (growth_rate BETWEEN 0 AND 0.5),
                    -- equivalent of Allow: Decimal, between 0 and 0.5

    account_code CHAR(10) NOT NULL,
                    -- fixed length, equivalent of Allow: Text length, = 10

    entry_date   DATE NOT NULL
                    CHECK (entry_date >= '2026-01-01')
                    -- equivalent of Allow: Date, greater than a fixed date
);

-- Attempting to INSERT a row violating any CHECK is REJECTED by the
-- database itself -- exactly Data Validation's Stop-style Error Alert,
-- except it is enforced centrally for every application that ever
-- writes to this table, not just for someone typing in one worksheet.
INSERT INTO scenario_inputs VALUES ('Optimistic Case', 0.30, 'ACC0001234'
    , '2026-08-01');
-- ERROR: new row violates check constraint on "scenario"
```

Key Concept

A SQL `CHECK` constraint and an Excel Custom Data Validation formula are doing precisely the same conceptual job: rejecting a write that violates a business rule, enforced as close to the point of entry as the tool allows. The meaningful difference is scope — a database constraint is enforced centrally, for *every* application and user that ever writes to that table, whereas an Excel validation rule only protects the specific cells it was applied to, and can be bypassed entirely by pasting data into a *different*, unvalidated cell and later copying it over — worth remembering before treating Data Validation as a full substitute for careful data governance in a widely shared workbook.

8 Worked Example: A Professional Assumptions Sheet in Three Tools

Everything in this module, combined into one small but genuinely professional artifact: a scenario-driven assumptions block that is optimised, named, validated, and correctly generalises — built identically in Excel, Python, and SQL.

8.1 The Brief

An "Assumptions" tab with: (1) a **Base Revenue** input and a **Tax Rate** input, each a Named Range; (2) a **Scenario** dropdown constrained by Data Validation to exactly three approved values, sourced from a Named Range; (3) a small two-way sensitivity grid showing projected after-tax profit across a range of revenue and growth-rate assumptions, built with correctly mixed references so one formula fills the entire grid; and (4) clean, gridline-free presentation formatting.

8.2 Excel Implementation

1. `B2` holds `100000`, named `BaseRevenue` (Name Box); `B3` holds `0.21`, named `TaxRate`.
2. `H2:H4` holds the three approved scenario names, named `ScenarioList` (Name Box, after selecting `H2:H4`).
3. `B4` gets Data Validation: **Allow: List**, **Source:** `=ScenarioList`, Error Alert style **Stop**, with a custom message naming the three valid options.

4. Row 6 (`C6:F6`) holds four growth-rate assumptions (`0%`, `5%`, `10%`, `15%`); column B (`B7:B10`) holds four revenue-multiplier scenarios. `C7` contains `=($B7*BaseRevenue)*(1+C$6)*(1-TaxRate)`, filled across `C7:F10` — one formula, correctly generalising across the whole grid via the mixed references `$B7` and `C$6`, exactly as in §5.

5. **View** → **Gridlines** unchecked; column B widened to fit `BaseRevenue`'s label; header row bolded.

Finance Application

Notice `TaxRate` is used with *no* `$` signs at all inside the grid formula, and still behaves as a fully absolute reference everywhere it's copied — this is the concrete payoff of §6's claim that named ranges default to absolute behaviour, removing the need to remember `$` for assumption cells at all.

8.3 Python Implementation

Listing 23: The full assumptions sheet, built end-to-end with openpyxl

```
from openpyxl import Workbook
from openpyxl.workbook.defined_name import DefinedName
from openpyxl.worksheet.datavalidation import DataValidation
from openpyxl.styles import Font

wb = Workbook()
ws = wb.active
ws.title = "Assumptions"
ws.sheet_view.showGridLines = False           # optimisation: clean presentation

# --- Inputs and named ranges ---
ws["A2"], ws["B2"] = "Base Revenue", 100000
ws["A3"], ws["B3"] = "Tax Rate", 0.21
ws["A4"], ws["B4"] = "Scenario", "Base Case"
wb.defined_names["BaseRevenue"] = DefinedName("BaseRevenue", attr_text="Assumptions!$B$2")
wb.defined_names["TaxRate"]     = DefinedName("TaxRate", attr_text="Assumptions!$B$3")

ws["H2"], ws["H3"], ws["H4"] = "Best Case", "Base Case", "Worst Case"
wb.defined_names["ScenarioList"] = DefinedName("ScenarioList", attr_text="Assumptions!$H$2:$H$4")

# --- Data validation, sourced from the named range, exactly like the Excel version ---
dv = DataValidation(type="list", formula1="=ScenarioList", allow_blank=False,
                    showErrorMessage=True, errorTitle="Invalid Scenario",
                    error="Choose Best Case, Base Case, or Worst Case.")
ws.add_data_validation(dv)
dv.add("B4")

# --- Two-way sensitivity grid: growth rates across, revenue multipliers down ---
growth_rates = [0.00, 0.05, 0.10, 0.15]
multipliers  = [0.8, 1.0, 1.2, 1.4]
for j, g in enumerate(growth_rates):
    ws.cell(row=6, column=3 + j, value=g)          # C6:F6
```

```
for i, m in enumerate(multipliers):
    ws.cell(row=7 + i, column=2, value=m)            # B7:B10

for i in range(4):
    for j in range(4):
        col_letter = chr(ord("C") + j)
        row_num = 7 + i
        # Same mixed-reference pattern as the Excel formula, written as a
            live string
        formula = f"=($B{row_num}*BaseRevenue)*(1+{col_letter}$6)*(1-
           TaxRate)"
        ws.cell(row=row_num, column=3 + j, value=formula)

ws["A1"].font = Font(bold=True)
wb.save("assumptions_sheet.xlsx")
```

8.4 SQL Implementation

Listing 24: The same assumptions, expressed as governed, constrained tables

```
CREATE TABLE assumptions (
    key   VARCHAR(20) PRIMARY KEY,
    value DECIMAL(12,4) NOT NULL
);
INSERT INTO assumptions VALUES ('BaseRevenue', 100000), ('TaxRate', 0.21)
    ;

CREATE TABLE scenario_selection (
    scenario VARCHAR(20) NOT NULL
             CHECK (scenario IN ('Best Case', 'Base Case', 'Worst Case'))
             -- equivalent of the List Data Validation sourced from
                ScenarioList
);
INSERT INTO scenario_selection VALUES ('Base Case');

CREATE TABLE sensitivity_inputs (
    growth_rate DECIMAL(5,4) NOT NULL,
    multiplier  DECIMAL(5,4) NOT NULL
);
INSERT INTO sensitivity_inputs VALUES
    (0.00, 0.8), (0.05, 0.8), (0.10, 0.8), (0.15, 0.8),
    (0.00, 1.0), (0.05, 1.0), (0.10, 1.0), (0.15, 1.0),
    (0.00, 1.2), (0.05, 1.2), (0.10, 1.2), (0.15, 1.2),
    (0.00, 1.4), (0.05, 1.4), (0.10, 1.4), (0.15, 1.4);

-- The grid itself: a CROSS JOIN against the single-row assumptions,
-- exactly the pattern introduced at the end of S4 for "one shared input
-- every row reaches back to" -- SQL's answer to a named, absolute
    reference.
SELECT
    si.multiplier,
    si.growth_rate,
    (si.multiplier * rev.value) * (1 + si.growth_rate) * (1 - tax.value)
        AS after_tax_profit
FROM sensitivity_inputs AS si
CROSS JOIN (SELECT value FROM assumptions WHERE key = 'BaseRevenue') AS
    rev
CROSS JOIN (SELECT value FROM assumptions WHERE key = 'TaxRate') AS tax;
```

9 Common Pitfalls & Debugging Checklist

Symptom	Likely cause & fix
Macro that worked yesterday now does nothing to new rows	Recorded against a fixed range (§4.7) — edit the code to compute the last row dynamically with `Cells(Rows.Count,"X").End(xlUp).Row` instead of a hard-coded address.
Macro is greyed out / workbook opened with a warning bar	Saved as `.xlsx` instead of `.xlsm` (macros silently stripped), or Trust Center macro settings are blocking execution (§4.5) — re-save as `.xlsm` and/or click Enable Content only if the source is trusted.
A formula gives a wildly different, but plausible-looking, result after being copied	A relative reference that should have been absolute or mixed drifted (§5) — press `F4` on the offending reference and re-check which of the four states is actually needed.
A sensitivity/lookup grid is correct in one row or column but wrong everywhere else	One of the two mixed-reference locks was set backwards or made fully absolute (§5) — re-derive which axis needs the row lock and which needs the column lock.
Every formula referencing a named assumption suddenly shows `#REF!`	The named range's source cell/row/column was deleted (§6) — open the Name Manager (`Ctrl+F3`) to find and fix or re-point the broken name.
A dropdown shows the right values on one sheet but a stale or wrong list on a copied sheet	A workbook-scoped name did not duplicate the way expected when the sheet was copied (§6) — verify in the Name Manager which sheet the name's **Refers to** field actually points to.
Old, clearly invalid data isn't circled even though Data Validation is applied	Validation only checks *new* entries by default (§7) — use **Circle Invalid Data** to retroactively flag pre-existing bad values.
A large workbook has become sluggish, freezing for seconds after every keystroke	Likely volatile functions (`OFFSET`, `INDIRECT`, whole-column references) or automatic recalculation of a large Data Table (§3.10) — switch to manual/Automatic-Except-Data-Tables calculation and audit for volatile functions.

10 Chapter Summary & Self-Check

Before moving to Module 3 (Excel Like a Pro), you should be able to, without looking anything up:

- ☐ Explain the difference between cosmetic and performance optimisation, and name at least three settings/techniques for each.
- ☐ Use Text to Columns to split delimited text and to fix numbers stored as text, and explain why a macro built on it can silently stop covering new rows.

- ☐ Record a macro with both relative and absolute reference recording, explain the difference, and know when each is the correct choice.
- ☐ Save a macro-enabled workbook correctly as `.xlsm` and explain why a plain `.xlsx` save silently discards macros.
- ☐ Read a recorded VBA macro line by line and identify exactly which hard-coded range would break it if the dataset grew, then fix it using `Cells(Rows.Count,"X").End(xlUp).Row`.
- ☐ Write all four reference types (`A1`, `$A$1`, `$A1`, `A$1`) from memory, and cycle through them with `F4` without thinking about it.
- ☐ Build a two-way sensitivity grid with a single formula using correctly mixed references, filled across the whole grid.
- ☐ Create a Named Range via all three methods, explain workbook vs. worksheet scope, and know what breaks (and what doesn't) when its source is deleted.
- ☐ Build a Data Validation List dropdown sourced from a Named Range, plus at least one Custom formula-based validation rule.
- ☐ Explain, for each of the five topics in this module, its closest Python and SQL equivalent, and articulate *why* several of these problems (reference drift especially) are largely unique to Excel's copy-based formula model.

11 What's Next: Module 3 Preview

Module 3, *Excel Like a Pro*, builds directly on the named ranges and validated inputs from this module and covers:

- **Find and Replace, Conditional Formatting** — locating and bulk-editing data, and visually flagging values automatically based on rules (a native alternative to the VBA highlighting macro built in §4, for cases simple enough not to need code at all).
- **IF / Nested IF / IFS** — branching logic inside a formula.
- **COUNTIF, AVERAGEIF, MAXIFS/MINIFS, SUMIF/SUMIFS** — conditional aggregation, directly building on this module's Data Validation-enforced clean categories (§7.4) to avoid the typo-driven under-counting pitfall flagged there.
- **Error Handling** — `IFERROR`, `IFNA`, and reading Excel's error codes (`#REF!`, `#N/A`, `#DIV/0!`, ...), including the `#REF!` case this module introduced via deleted named ranges.
- **VLOOKUP, HLOOKUP, XLOOKUP, INDEX/MATCH** — retrieving data by a lookup key rather than a hard-coded position, replacing the last major category of positional fragility not yet solved by named ranges or Tables.
- **String Functions** — `LEFT`, `RIGHT`, `MID`, `FIND`, `CONCATENATE`/`TEXTJOIN`, and others, extending the manual, one-time Text to Columns split from §3.3 into live, formula-driven text manipulation.